MODULE 07 · 30 MIN

Multimodal: Screenshot to UI

Claude can read a picture. Hand it a wireframe; get a working UI back.

Theory · Layout-first prompting (4 min)

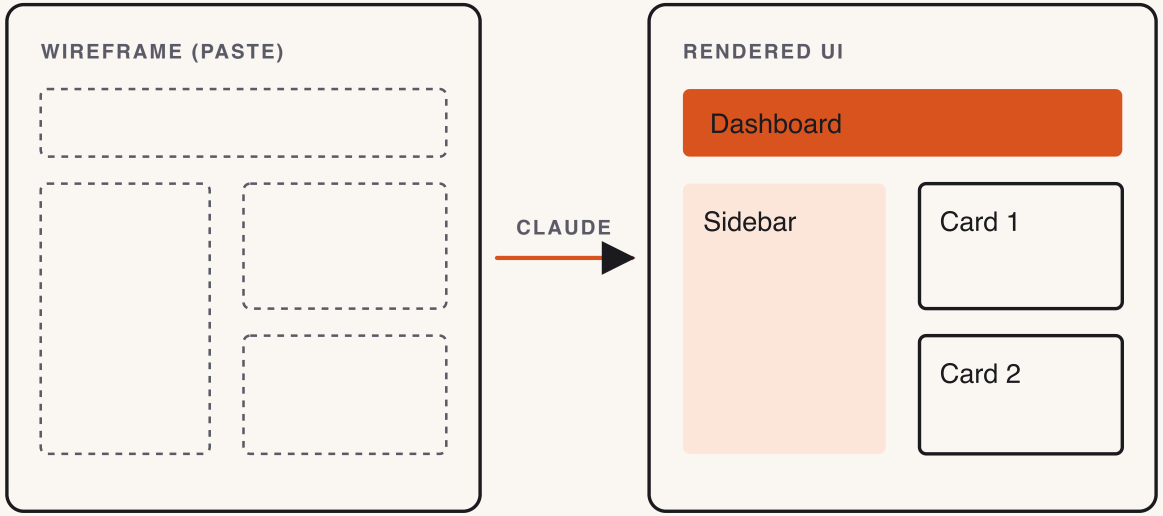
> Let Claude **read the layout** from the image; you describe what it **can't** see.

- The image carries: structure, regions, relative sizes.
- You must state: framework, data source, interactivity — Claude can't infer these.
- **Visual-diff loop**: render → screenshot → ask Claude "what's missing?" → patch. **Cap at 3 rounds.**
- **Scope discipline**: ship the layout. Theming and animation are stretch goals.

Two wireframes ship with the exercise: `wireframe.png` (canonical) and `wireframe-sketch.png` (rough).

From wireframe to running UI

IMAGE → PROMPT → WORKING UI



Layout-first prompt → build → **screenshot-diff loop** (cap at 3 rounds).

Reference · markitdown — any file → Markdown

For **non-image** sources (PDF, DOCX, PPTX, XLSX, audio, video, HTML, ZIP, YouTube), convert to Markdown first — it's cheap and LLM-native:

```
pip install 'markitdown[all]'
markitdown report.pdf > report.md
```

Then drop into the prompt: "*Attached is the converted Markdown of* `report.pdf`*.*" Claude consumes tables and headings without burning vision tokens.

Reference · Common mistakes

- "Looks close enough" — the whole point is precision; diff again.
- Pulling in Tailwind / shadcn (the constraint exists for a reason).
- Forgetting to attach the image.
- Iterating five rounds (cap at three).

Live demo · Wireframe → running UI (6 min)

1. Open `exercises/part-07/wireframe-sketch.png` in Claude Code.
2. Paste the prompt **with the framework constraint**:

```
Build this wireframe as a Flask + Jinja app (no other deps): one route, one
template. Match the layout — header, sidebar, main, footer. Run on localhost:5000.
```

3. Save and run; screenshot it next to the wireframe, ask "*What's missing?*"
4. Apply one round of fixes; end on a side-by-side comparison.

Success signal: the app runs with one command and the layout clearly matches the wireframe.

Your turn · Dashboard from wireframe (13 min)

Exercise: `exercises/part-07/README.md`

Build a single-page dashboard matching the wireframe (static data OK):

```
Header (title + primary action) · Sidebar (3–5 nav links)
Main (3 KPI cards + table of 5 rows) · Footer (version string)
```

Run the **visual-diff loop** at least once; record patches in `diff-notes.md`.

Deliverables: runnable app in `module-07/` · `render-final.png` at 1280×720 · `diff-notes.md`.

Success signal: render at 1280×720 unmistakably matches the wireframe.

✓ Done & next (1 min)

Definition of done

- ☐ [ ] Runnable app; header, sidebar, 3 KPI cards, 5-row table, footer all present.
- ☐ [ ] `render-final.png` at 1280×720.
- ☐ [ ] `diff-notes.md` records ≥ 1 visual-diff round.

Next — we take messy code and make it clean *under constraints*, then document it.

Module 8 — Refactoring & Documentation at Scale.